

Decision-Focused Learning for Berth Allocation

Puerto de San Antonio, Chile | meeting walkthrough

These figures explain paper 3 of the dissertation. Assumes optimization background; no prior DFL exposure required. All formulas trace to the code in `prediction_models/`.

Contents.

1. Figure 2. The MILP we solve.
2. Figure 2c. The classical discrete BAP from the literature.
3. Figure 2d. Alternatives to DFL.
4. Figure 4. PtO, DFL, and how we score them (*the main figure*).
5. Figure 5b. How the test problems are built.
6. Figure 5c. The blackbox gradient (Pogančić et al., 2020).
7. Figure 5d. The Gaussian-perturbed gradient (Berthet et al., 2020).
8. Figure 6. Where DFL pays off.

Figure 2. The MILP we solve

The berth-allocation MILP that runs inside every training step and at evaluation time. Source: `prediction_models/src/ports_dfl/optim/discrete_bap.py`.

Sets. $\mathcal{I} = \{1, \dots, N\}$ vessels; $\mathcal{B} = \{1, \dots, B\}$ berths.

Parameters. a_i arrival time of vessel i ; $w_i \geq 0$ priority weight; $\tau_i \geq 0$ service time (the predictor’s output, plugged in as a mutable parameter); M big-M (horizon plus a slack so any single-berth schedule fits).

Decision variables. $x_{ib} \in \{0, 1\}$ is 1 iff vessel i is processed at berth b ; $s_i \in \mathbb{R}_{\geq 0}$ start time of vessel i ; $z_{ijb} \in \{0, 1\}$ is 1 iff i precedes j at berth b (defined for ordered pairs $i \neq j$).

Expression		Meaning
$\min_{x, s, z}$	$\sum_{i \in \mathcal{I}} w_i (s_i + \tau_i)$	total weighted completion time
s.t.	$\sum_{b \in \mathcal{B}} x_{ib} = 1$	each vessel assigned to exactly one berth
	$s_i \geq a_i$	no vessel starts before it arrives
	$z_{ijb} + z_{jib} \leq 1$	at most one direction of precedence per pair-berth
	$z_{ijb} + z_{jib} \geq x_{ib} + x_{jb} - 1$	two vessels sharing a berth must be sequenced
	$s_j \geq s_i + \tau_i - M(1 - z_{ijb})$	big-M precedence: if $z_{ijb}=1$, j starts after i finishes
	$x_{ib}, z_{ijb} \in \{0, 1\}, s_i \geq 0$	

Where τ enters. τ appears in both the objective and the precedence constraints. At training time the solver receives $\hat{\tau} = f_{\theta}(x)$ in place of τ ; at evaluation time start times are recomputed under the true τ on the locked (x, z) decision, so regret is always non-negative.

Notes. In scheduling terms this is $P|r_j|\sum w_j C_j$ in Graham notation: berths as identical parallel machines, arrivals as release dates, no due dates. Solved with Gurobi via Pyomo; τ is a mutable Pyomo **Param** so each DFL training step updates values rather than rebuilding the model.

Figure 2c. The classical discrete BAP from the literature

What the operations-research literature actually calls the discrete dynamic Berth Allocation Problem (DBAP). Reference formulation: Cordeau, Laporte, Legato & Moccia, “Models and tabu search heuristics for the berth-allocation problem,” *Transportation Science* 39(4):526–538, 2005 (“DBAP1”). Compared with Figure 2, the classical model differs in four places, each highlighted below.

Sets. $\mathcal{I} = \{1, \dots, N\}$ vessels; $\mathcal{B} = \{1, \dots, B\}$ berths.

Parameters. a_i arrival time; b_i departure deadline (time window) [NEW]; $w_i \geq 0$ priority weight; $\tau_{ib} \geq 0$ service time of vessel i at berth b [BERTH-DEPENDENT, NEW]; M big-M constant.

Decision variables. $y_{ib} \in \{0, 1\}$ vessel–berth assignment; $T_i \in \mathbb{R}_{\geq 0}$ berthing (start) time; $D_i \in \mathbb{R}_{\geq 0}$ departure time; $\sigma_{ij} \in \{0, 1\}$ cross-berth precedence: 1 iff i finishes before j starts, regardless of berth [DIFFERENT VARIABLE, CLASSICAL CHOICE].

Expression		Range
$\min_{y, T, D, \sigma} \sum_{i \in \mathcal{I}} w_i (D_i - a_i)$		total weighted time in port [NEW OBJECTIVE]
s.t. $\sum_{b \in \mathcal{B}} y_{ib} = 1$	$\forall i$	
$T_i \geq a_i$	$\forall i$	
$D_i = T_i + \sum_{b \in \mathcal{B}} \tau_{ib} y_{ib}$	$\forall i$	[BERTH-DEPENDENT τ]
$D_i \leq b_i$	$\forall i$	[TIME WINDOW, NEW]
$T_j \geq D_i - M(3 - \sigma_{ij} - y_{ib} - y_{jb})$	$\forall i \neq j, b \in \mathcal{B}$	
$\sigma_{ij} + \sigma_{ji} \leq 1$	$\forall i < j$	
$y_{ib}, \sigma_{ij} \in \{0, 1\}, T_i, D_i \geq 0$		

Differences from Figure 2 (the MILP we solve).

1. **Service time depends on the berth** (τ_{ib} , not τ_i). Captures berth-specific productivity, crane count, draft, and equipment. Turns identical parallel machines into unrelated parallel machines: in Graham notation the problem jumps from $P | r_j | \sum w_j C_j$ to $R | r_j | \sum w_j C_j$.
2. **Time windows** ($D_i \leq b_i$). Vessels have departure deadlines; infeasibility is possible, and tardiness penalties often replace the hard constraint in practice.
3. **Cross-berth precedence** σ_{ij} instead of per-berth z_{ijb} . Fewer binaries (N^2 vs. $N^2 B$) but a looser big-M coupling ($3 - \sigma - y - y$ instead of $1 - z$). Equivalent expressivity.
4. **Time-in-port objective** $\sum w_i (D_i - a_i)$ rather than weighted completion time $\sum w_i (s_i + \tau_i)$. The two differ only by the constant $\sum w_i a_i$, so the optimal decision is identical; the reported number is in “extra time spent in port” rather than “absolute clock time of completion.”

Things the classical DBAP still does not include (and that are added in richer variants):

- *Spatial dimension*: continuous BAP treats the quay as a line and vessels as length-occupying intervals (2D packing in space $\times$ time).
- *Tides and draft-time profiles*: berth availability varies with the tidal cycle.
- *Quay-crane assignment (BAP+QCAP)*: cranes are shared across berths and themselves need scheduling.
- *Stochastic arrivals or service times*: classical DBAP is deterministic. This is the gap the present project addresses by predicting τ from data.

The reference discrete dynamic BAP from the literature. Compared with Figure 2, it adds berth-dependent service time, departure deadlines, and a cross-berth precedence variable. The MILP we solve omits these on purpose so the predictor has a single learned quantity (vessel service time) and the DFL signal stays clean.

Figure 2d. Alternatives to DFL

Four other paradigms handle unknown service times in the classical BAP. They differ in when they confront the uncertainty and what they assume about it.

1. Robust optimization (RO).

- Bound each service time inside a plausible range; pick the schedule that performs best against the worst case.
- Worst-case guarantees but conservative: berths sit idle absorbing the bounds. The predictor only sizes the bounds, it does not guide the schedule.
- *Refs*: Ben-Tal, El Ghaoui & Nemirovski (2009); Bertsimas & Sim (2004).

2. Stochastic programming (SAA).

- Sample many service-time scenarios from a predictive distribution; minimize average cost across scenarios in one large MILP.
- **Bayesian source.** The hierarchical models in `bayesian_model/` (M2 heavy-tail, M3 heteroscedastic) are the natural scenario source: each posterior draw is one vessel-correlated scenario.
- MILP size grows linearly with scenario count; solution quality is bounded by scenario fidelity.
- *Refs*: Birge & Louveaux (2011); Karam & Eltawil (2016).

3. Distributionally robust optimization (DRO).

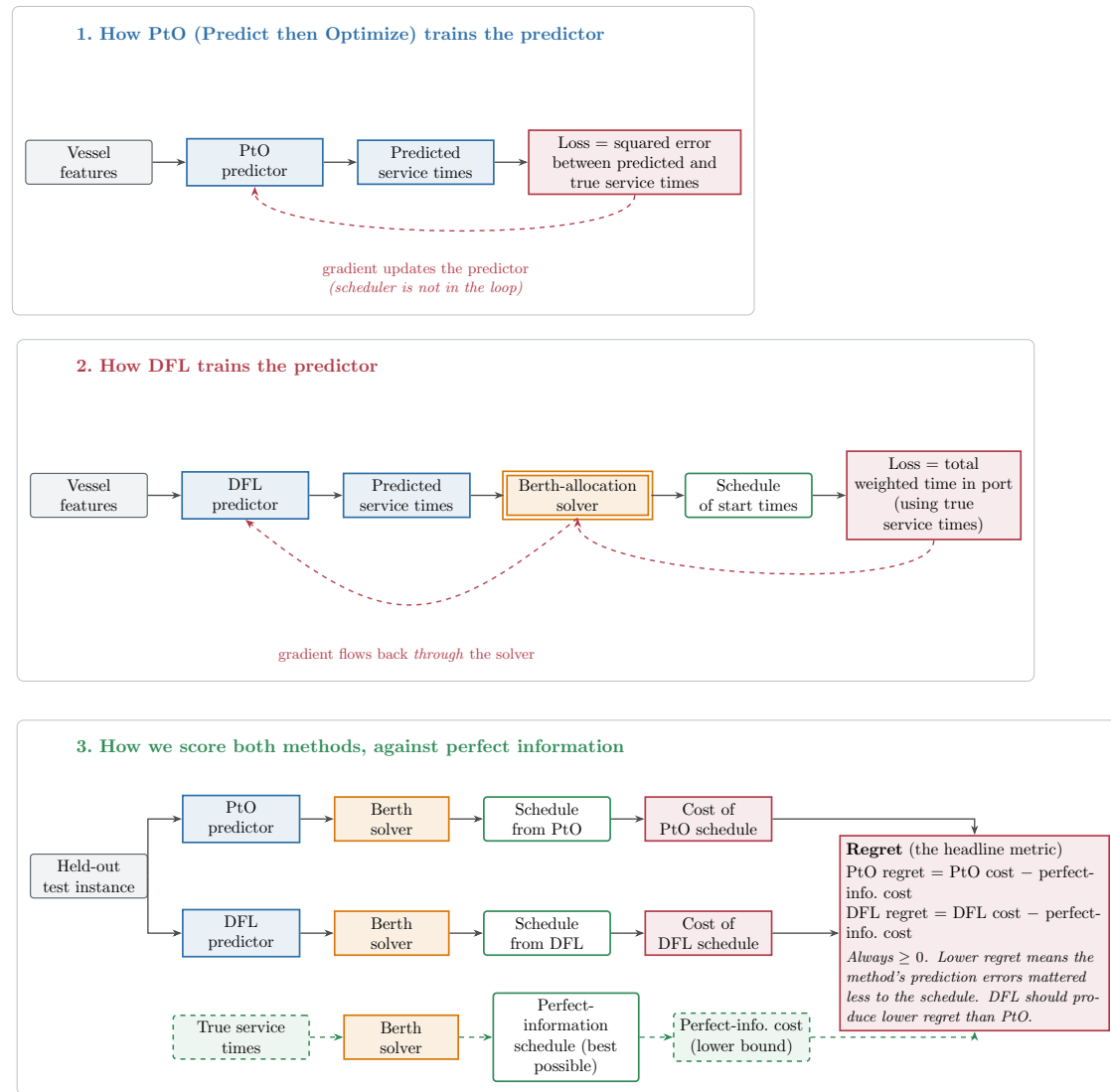
- Pick the schedule that performs best against the worst plausible distribution from a family centred on the data.
- **Bayesian source.** The same hierarchical models supply both the centre (posterior predictive) and the radius (posterior spread), so SAA and DRO share one back-end.
- Less conservative than RO; tractability depends on the family of distributions chosen.
- *Refs*: Esfahani & Kuhn (2018); Delage & Ye (2010).

4. End-to-end neural scheduler.

- Drop the predictor and the MILP. Train a neural network to output schedules directly from features (reinforcement learning with port cost as reward, or imitation of MILP solutions).
- Millisecond inference, but no feasibility guarantees, brittle under distribution shift, loses MILP interpretability.
- *Refs*: Kool et al. (2019); Bello et al. (2017).

Four alternative paradigms for unknown service times. RO worst-cases over bounds; SP averages over scenarios; DRO hedges over distributions; end-to-end ML replaces the MILP entirely. DFL (this paper, Figure 4) keeps the MILP and trains the predictor to make its decisions good. The Bayesian hierarchical models in `bayesian_model/` feed SAA scenarios, DRO ambiguity radii, and RO bounds from a single back-end.

Figure 4. PtO, DFL, and how we score them

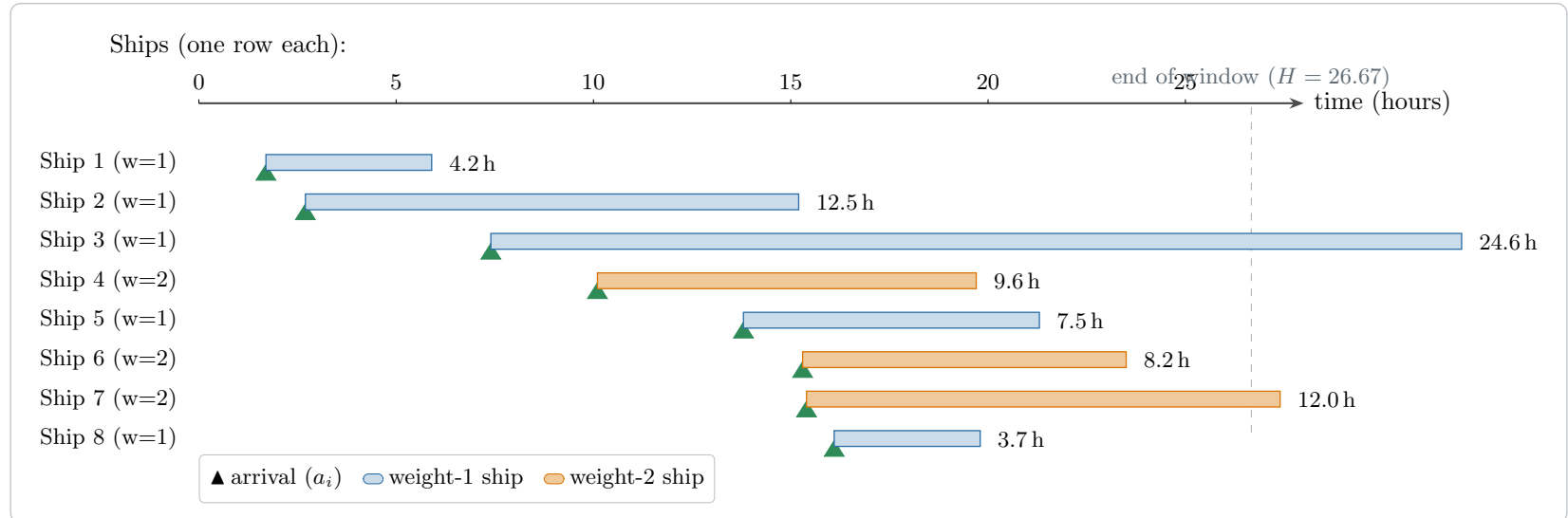


The whole story in one picture. Bands 1 and 2 are the two ways to *train* the predictor. The only difference between them is whether the berth scheduler participates in training. Band 3 is how we *score* both methods after training, against a perfect-information solver that already knew the answer. Lower regret wins.

Figure 5b. How the test problems are built

One test problem = one day at a small port. Each instance is a ≈ 27 -hour scheduling window with 8 ships and 3 berths. The 8 ships are not all known up front — each one has its own arrival time within the day, and the scheduler decides which berth to send it to and when to start serving it. The unknown the predictor learns is each ship’s *service time* (how long it occupies the berth). The numbers in the table below are the knobs of the synthetic generator (`ports.dfl.optim.classic.bap.make_classic_problem`); the diagram below the table shows what one realized instance actually looks like.

Parameter	Value	Meaning
Ships per problem (N)	8	vessels to be scheduled in this window
Berths (M)	3	parallel berths available
Scheduling window (H)	≈ 27 h	one day; set so total expected work $\approx$ total berth-hours
Contention	1.0	expected total work / total berth capacity
Arrival distribution	uniform on $[0, 0.7H]$	ships arrive over the first ≈ 19 h of the window
Service-time distribution	lognormal, mean 10 h	σ in log-space = 0.5 ($\approx 50\%$ coefficient of variation)
Priority weights	three classes $\{1, 2, 5\}$	probabilities $\{60\%, 30\%, 10\%\}$; weight 5 ships count $5\times$ in cost
Features per ship	10	1 noisy estimate of τ + 1 priority weight + 8 random-noise features
Training problems per seed	80	used to fit the predictor
Test problems per seed	60	used to measure regret
Random seeds	4 (1, 2, 3, 42)	240 test problems pooled across seeds



One realized instance from the synthetic generator (seed 42, first test problem). *Top axis*: the 27-hour scheduling window. *Each row*: one ship. The green triangle marks when the ship arrives at the port. The bar drawn to its right is the ship’s true service time — the number of hours that ship will occupy a berth once it starts being served. Bars are drawn starting at the arrival time only to make their widths visible; the scheduler is free to delay the start of any ship and to choose any of the three berths. The total bar-length sums to about 82 ship-hours and the three berths offer 80 berth-hours over the window, hence contention ≈ 1.0 . In this particular seed there are three weight-2 ships and five weight-1 ships; weight-5 ships have a 10% probability so a typical 8-ship instance has 0–1 of them.

Figure 5c. The blackbox gradient (Pogančić et al., 2020)

The problem. The scheduler turns predicted service times into a discrete schedule (which ship at which berth, in what order, starting when). Discrete output means small changes in the input either leave the schedule unchanged or flip it abruptly — the “true gradient” of the schedule with respect to the predictions is zero almost everywhere and undefined at the jumps. Neither helps training a neural network.

The recipe. Pogančić et al. get a usable gradient by calling the scheduler *twice* and taking a finite difference between the two answers:

$$\begin{array}{ll} \mathbf{s}^* = \text{scheduler}(\hat{\boldsymbol{\tau}}) & \text{(first solve)} \\ \mathbf{g} = \partial L / \partial \mathbf{s}^* & \text{(loss-gradient w.r.t. the schedule)} \\ \mathbf{s}^{*'} = \text{scheduler}(\hat{\boldsymbol{\tau}} + \lambda \mathbf{g}) & \text{(second solve, perturbed)} \\ \frac{\partial L}{\partial \hat{\boldsymbol{\tau}}} \approx -\frac{1}{\lambda} (\mathbf{s}^* - \mathbf{s}^{*'}) & \text{(surrogate gradient used for backprop)} \end{array}$$

What every symbol means.

- $\hat{\boldsymbol{\tau}}$: the model’s predicted service times (one number per ship in the instance).
- $\mathbf{s}^*, \mathbf{s}^{*'}$: two schedules (vectors of start times). The first comes from the original predictions; the second from slightly nudged predictions.
- $\mathbf{g}$: how the loss would change with each ship’s start time. Computed by standard chain rule once the loss is defined — it tells us *which direction* we’d like the schedule to move.
- λ : the perturbation size, a fixed positive number (here $\lambda = 5$). The only hyperparameter of the method.

Why this works (intuition). The perturbed cost $\hat{\boldsymbol{\tau}} + \lambda \mathbf{g}$ pushes the scheduler in the direction the loss says we’d like the schedule to move. The change in the schedule between the two solves, divided by λ , is a finite-difference estimate of “how the schedule depends on $\hat{\boldsymbol{\tau}}$.” That is the gradient we need for backpropagation.

Hyperparameter trade-off. A small λ gives a sharper (less biased) gradient but more variance from individual solver corners; a large λ is smoother but biased because it averages over more discrete jumps. The method costs exactly **two scheduler calls per training step**, regardless of the problem size.

Pogančić et al. (2020) build a finite difference of the scheduler’s output between the original predictions and a single targeted perturbation. The result is the gradient used to train the predictor.

Figure 5d. The Gaussian-perturbed gradient (Berthet et al., 2020)

Same problem, different fix. Berthet et al. smooth the scheduler by averaging its output over *many* random perturbations. The averaged output is a smooth function of the predictions, with a clean gradient formula.

The recipe. Choose a number of samples K and a noise magnitude σ . For each training step:

$$\begin{aligned}
 \varepsilon_{k,i} &\sim \mathcal{N}(0, \sigma^2) \text{ independently for each } k = 1, \dots, K \text{ and ship } i = 1, \dots, N \\
 \mathbf{s}_k &= \text{scheduler}(\hat{\boldsymbol{\tau}} + \boldsymbol{\varepsilon}_k) && \text{(solve once per sample)} \\
 \bar{\mathbf{s}} &= \frac{1}{K} \sum_{k=1}^K \mathbf{s}_k && \text{(smoothed schedule, the forward pass)} \\
 \frac{\partial L}{\partial \hat{\boldsymbol{\tau}}} &\approx \frac{1}{K\sigma^2} \sum_{k=1}^K L(\mathbf{s}_k) \boldsymbol{\varepsilon}_k && \text{(surrogate gradient used for backprop)}
 \end{aligned}$$

What every symbol means.

- $\hat{\boldsymbol{\tau}}$: predicted service times (same as before).
- $\boldsymbol{\varepsilon}_k$: a random noise vector — one normal random number per ship, drawn independently with mean 0 and standard deviation σ . A new vector is drawn for each of the K samples in a step.
- σ : the noise magnitude (here $\sigma = 1$ h, same units as the predicted service times). Controls how far the perturbed predictions wander from the originals.
- K : the number of perturbation samples (here $K = 10$). Controls how noisy the gradient estimate is.
- $\mathbf{s}_k$: the schedule produced at the k -th perturbed input.
- $L(\mathbf{s}_k)$: a scalar — the loss evaluated at the schedule from sample k .

Why this works (intuition). Adding Gaussian noise smears out the scheduler’s discrete jumps: $\bar{\mathbf{s}}$ moves smoothly as $\hat{\boldsymbol{\tau}}$ moves, even though each individual $\mathbf{s}_k$ is discrete. The gradient formula is a *score-function* (REINFORCE-style) estimator: it weights each noise direction by how good or bad the resulting schedule was, so on average the predictor learns to shift $\hat{\boldsymbol{\tau}}$ toward predictions that produce low-cost schedules.

Hyperparameter trade-off.

- K : more samples $\Rightarrow$ a lower-variance gradient, but the training step now costs K scheduler calls (here $\approx 10\times$ more than the blackbox method).
- σ : larger noise $\Rightarrow$ a smoother gradient but more bias (the smoothed schedule is further from the true argmin).

Compared with the blackbox method. The blackbox method makes *one* targeted perturbation in the direction the loss says we want the schedule to move; Berthet’s method makes *many random* perturbations and lets the averaging do the smoothing. Berthet is unbiased in the limit $K \rightarrow \infty$ (a Monte-Carlo estimate); blackbox is a deterministic finite-difference approximation.

Berthet et al. (2020) smooth the scheduler by averaging its output over many Gaussian-perturbed inputs. The gradient is the loss-weighted average of the noise directions — predictions that lead to lower-cost schedules are reinforced.

The two hyperparameters are σ and K . The noise itself has no other knobs — each ship gets an independent draw from the same normal distribution, with no covariance between ships. The only choices are the noise magnitude σ and the number of samples K .

How σ and K are tuned in practice.

- **Default values.** The PyEPO library (the de-facto standard implementation) ships with $\sigma=1$ and $K=10$. Used as-is in many DFL papers and in this project.
- **Scale σ to the cost vector.** σ is added to $\hat{\tau}$ in the same units as $\hat{\tau}$. Practitioners either (i) standardize the cost so $\sigma=1$ is meaningful or (ii) sweep $\sigma \in \{0.1, 0.5, 1, 2\}$. Too small a σ leaves the gradient zero (no perturbed sample crosses a scheduler decision boundary); too large washes out the signal.
- **Pick σ by grid search on validation regret.** Mandi et al.’s JAIR 2024 DFL survey reports this as the most common protocol. Bayesian tuning is rare.
- **K trades variance for solver calls.** Variance of the gradient $\propto 1/K$. The training step costs K scheduler calls. Common values are $K \in \{1, 3, 10\}$; expensive solvers favour small K , cheap solvers favour large K .
- **Recipe.** Pick the smallest σ that gives a non-zero gradient on most samples, then raise K only until the validation regret stops improving.

Picking σ from the BAP structure. The rules above are problem-agnostic — they tune σ against training stability, not against the BAP’s own decision granularity. When the BAP carries time windows, the perturbation has a natural operational scale: the *slack* between a vessel’s predicted completion and its deadline. Perturbations smaller than the slack rarely flip a decision; perturbations larger than the slack reliably do. Three rules tie σ to that structure:

- **Median window width.** $\sigma = \text{median}_i (b_i - a_i)$ across the training instances. One scalar, fixed across training; reads as “size the perturbation to the typical scheduling window.” Cleanest option for a paper headline.
- **Median slack.** $\sigma = \text{median}_i (b_i - a_i - \hat{\tau}_i)$, recomputed per instance. Adapts to the tightness of the current problem — σ shrinks when windows are crowded, grows when they are loose.
- **Per-vessel slack (anisotropic).** $\sigma_i = \alpha \cdot \text{slack}_i$ with $\alpha \in [0.5, 1]$. Each ship gets noise proportional to its own room before the deadline: ships near their cliffs get small, targeted perturbations; ships with room to spare get broad ones. Floor σ_i at a small positive value when $\text{slack}_i \leq 0$ so the vessels DFL most needs to learn from are not skipped. Loses Berthet’s clean isotropic theory but matches the BAP’s per-vessel heterogeneity.

Justification. Berthet’s σ is correct when it matches the scale of the LP’s piecewise-linearity. In BAP with time windows that scale is set by the deadline slack: cost cliffs sit exactly at $s_i + \tau_i = b_i$, so the right smoothing radius is the typical distance to that boundary.

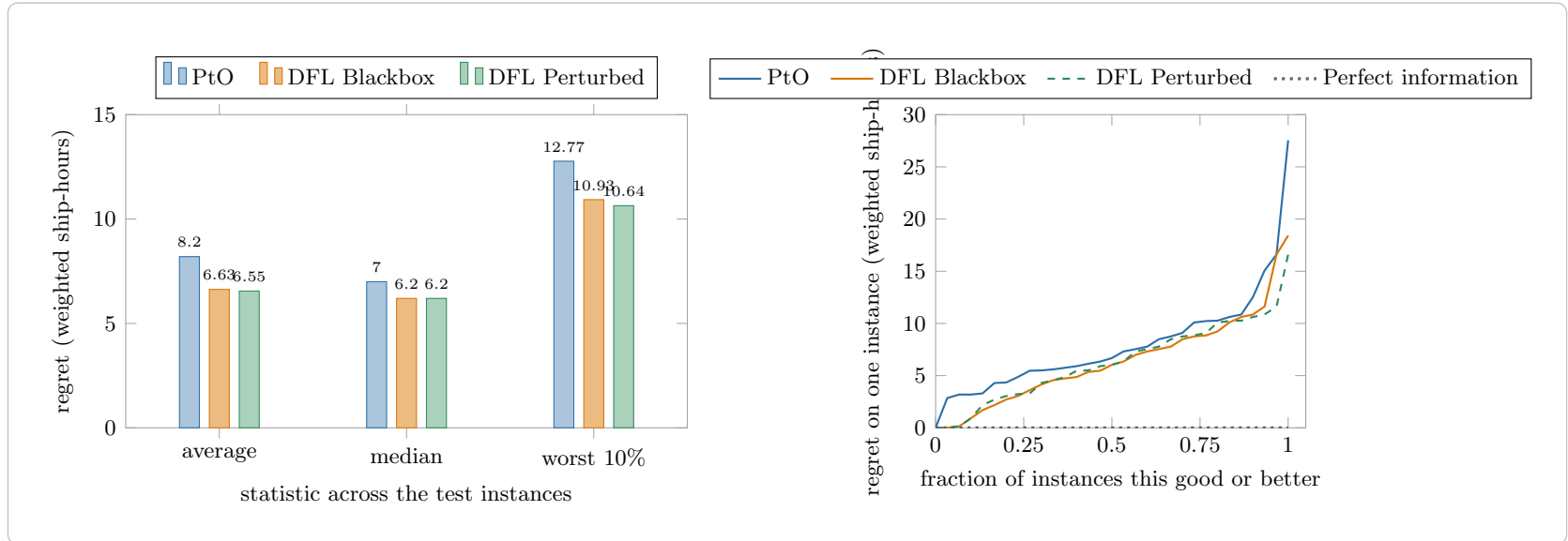
Sources. Berthet et al., NeurIPS 2020 (arXiv:2002.08676); PyEPO source `pkg/pyepo/func/perturbed.py`; Tang & Khalil, *PyEPO* (MPC 2024, arXiv:2206.14234); Mandi et al., JAIR 2024 DFL survey (arXiv:2307.13565).

Figure 6. Where DFL pays off (linear regression predictor trained with schedule-cost gradients)

What we plot. For each test instance (8 ships to schedule on 3 berths) we compare two costs: the cost of the schedule the method’s predictions produce, and the cost a perfect-information scheduler (one that already knew the true service times) would have produced. The gap between the two is the *regret*. Low regret means the method’s prediction errors did not really hurt the schedule. Regret is measured in weighted ship-hours, the same units as the scheduler’s objective: each ship’s completion time multiplied by its priority weight.

Setup. 8 ships per instance, 3 berths, three-class priority weights $\{1, 2, 5\}$ (about 10% of ships are $5\times$ as important as the rest). Three methods, all using the same linear predictor and the same training data:

- **PtO** – predictor trained on prediction error (the standard recipe).
- **DFL Blackbox** – predictor trained on schedule cost, gradient via Pogančić et al. (2020).
- **DFL Perturbed** – predictor trained on schedule cost, gradient via Berthet et al. (2020).



Notation cheat-sheet

Symbol	Meaning
τ_i	true service time for vessel i (target, unknown ex ante).
$\hat{\tau}_i$	predicted service time from f_θ .
x_{ib}	binary, 1 if vessel i is assigned to berth b .
s_i	start time of vessel i at its berth ($s_i \geq a_i$).
z_{ijb}	binary precedence: 1 if i precedes j at berth b .
a_i	arrival time of vessel i .
w_i	priority weight of vessel i .
λ	blackbox interpolation strength; $\lambda=5$ here.
FI	full-information schedule: MILP on the true τ .
s^*	start times from the MILP at $\hat{\tau}$ (training time).
f_θ	predictor; here <code>nn.Linear($d, 1$)</code> (same architecture in both).
L_{PtO}	$\ \hat{\tau} - \tau\ _2^2$.
L_{DFL}	$\sum_i w_i(s_i^* + \tau_i)$.
$C_{\text{PtO}}, C_{\text{DFL}}, C_{\text{FI}}$	evaluation-time schedule costs (always on true τ).
Regret	$C_{\text{method}} - C_{\text{FI}} \geq 0$. Lower is better.

Source code, all under `prediction_models/`:

- `src/ports_dfl/optim/discrete_bap.py` (MILP, objective, big-M, mutable τ).
- `src/ports_dfl/optim/classic_bap.py` (synthetic instance generator used in Figure 5b).
- `src/ports_dfl/train/pto.py` (L_{PtO} , AdamW + Cosine).
- `src/ports_dfl/train/dfl_blackbox.py` (Pogančić blackbox gradient, λ).
- `src/ports_dfl/train/dfl_perturbed.py` (Berthet Gaussian-perturbed gradient, σ, K).